

Appendix D. Data design

AERO-502 / Sprint 1 / September 23, 2026

What changed

The earlier design stored actual hours on TASK. That can show a total, but it cannot tell us who worked, on which date, or which entry a member may correct. TIME_ENTRY now holds those facts. Actual totals are calculated from those entries.

TASK now has a start date. TASK_STATUS_EVENT records successful status changes. TASK_LINK allows more than one reference link without putting a list into one field. Team roles remain on TEAM_MEMBERSHIP, and the Chief Engineer role remains on USER because those permissions have different scopes.

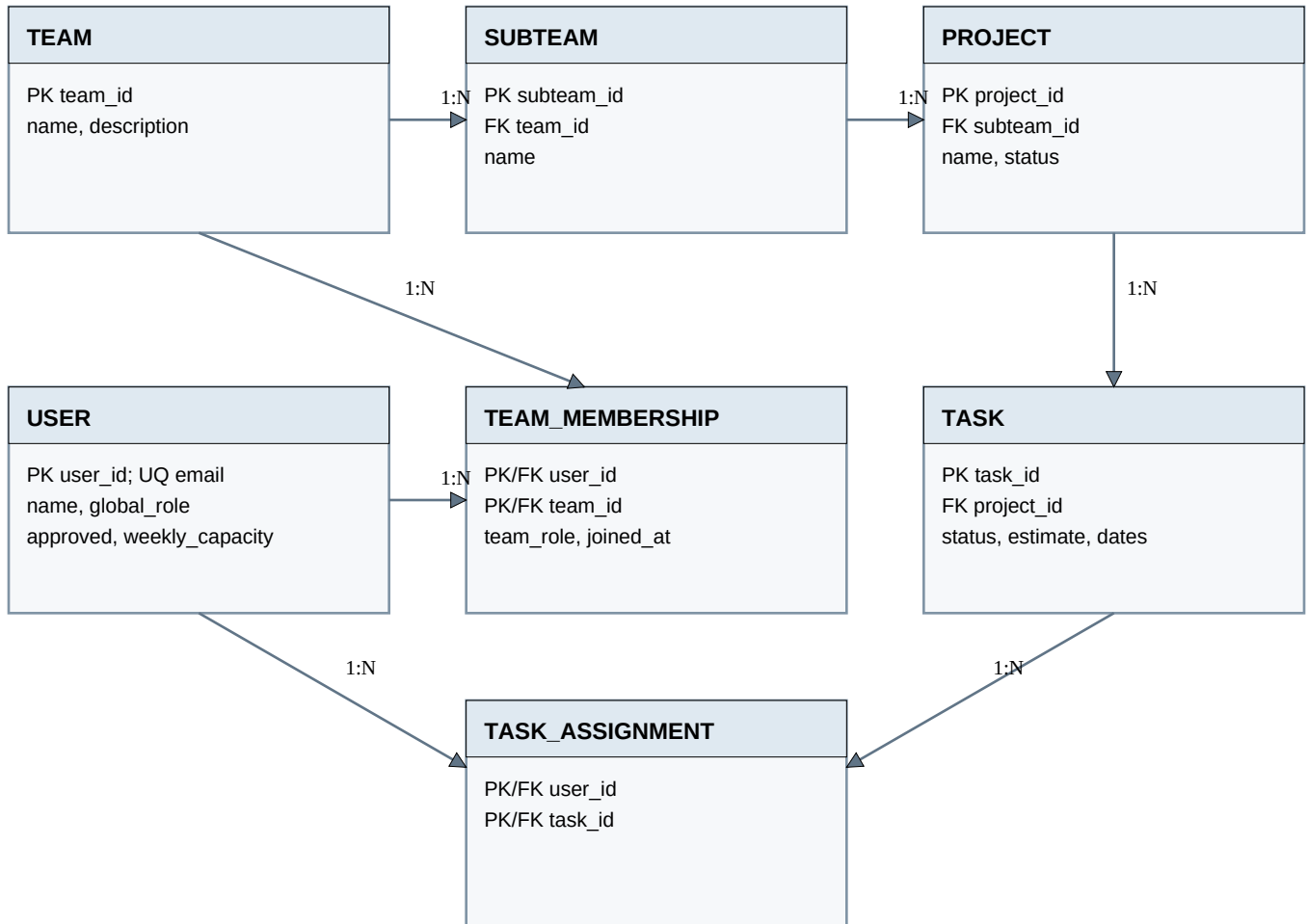
The diagrams on the following pages show required foreign keys and one-to-many relationships. A child has exactly one parent unless its foreign key is marked nullable. A parent may have zero or more children. PK means primary key, FK means foreign key, and UQ means unique.

Access rules

Account	Allowed actions
Chief Engineer / organization administrator	View all teams. Create tasks, assign members, edit estimates, view dashboards and change team roles.
Officer / Team Lead	View and manage tasks and estimates within teams where the user is an officer. View those teams' dashboards. Cannot change another user's role.
Member	View assigned tasks, change their status and log hours. Edit/delete only their own time entries. Cannot create tasks, reassign work, edit estimates, view leader dashboards or change roles.

USER.global_role is chief_engineer or standard. TEAM_MEMBERSHIP.team_role is officer or member. An officer of Team A may be a member of Team B. Membership is unique for each user/team pair. A Chief Engineer's organization permission is checked before team permissions. Role changes are authorized on the server and take effect on the next request/page load. An unapproved account cannot access organization records.

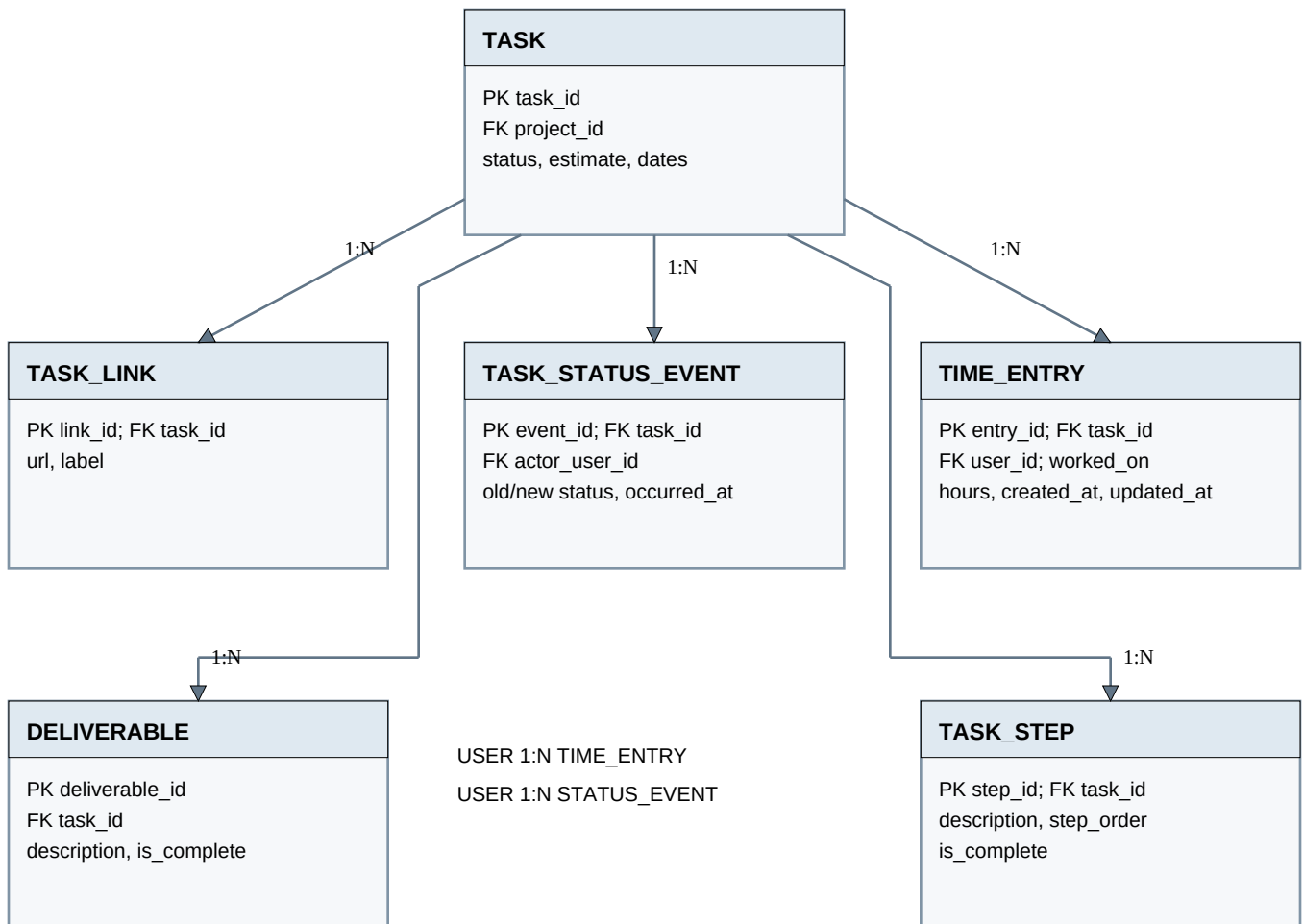
Core relationships



TEAM to SUBTEAM to PROJECT to TASK follows the existing design. The task form derives the team/subteam from its project and rejects inconsistent choices. This avoids storing separate team and subteam values on TASK that could disagree with its project. Sprint 1 uses seeded organization records; S01 adds their management screens in Sprint 2.

The two membership tables resolve many-to-many relationships. Their composite keys prevent duplicate team memberships and task assignments. A task may have several assignees; each assignment still refers to one task and one user.

Task details, hours and history



TIME_ENTRY uses its own key because a member can make several entries for the same task/date. Removing an assignment does not delete past time entries. Authorization checks the current assignment before accepting new hours. An entry's user is taken from the signed-in account, not a user ID supplied by the form.

TASK_STATUS_EVENT is written in the same transaction as a successful status change. If either write fails, both roll back. Rejected changes do not create success events. The application exposes no edit/delete action for these events, including to the Chief Engineer. Tasks and users referenced by history are retained or deactivated rather than cascade-deleted. Event time is stored in UTC and displayed with its timezone.

Sprint 2 adds TASK_ASSIGNMENT_EVENT (event_id PK, task_id FK, actor_user_id FK, target_user_id FK, action, occurred_at). Action is assigned or unassigned. Its screen combines assignment events with Sprint 1 status events, orders by time and then event ID, and applies team-level read permissions. This later table is not a prerequisite for Sprint 1 status capture.

Fields and constraints

Table / fields	Rule
USER	user_id PK; name; email UQ; global_role; approved boolean; weekly_capacity numeric, nullable and nonnegative. Authentication identity fields follow the chosen sign-in provider.
TEAM / SUBTEAM / PROJECT	Each has an ID PK and nonempty name. SUBTEAM.team_id and PROJECT.subteam_id are required FKs. TEAM retains description; PROJECT retains status.
TEAM_MEMBERSHIP	(user_id, team_id) composite PK and FKs; team_role required; joined_at required.
TASK	task_id PK; project_id FK; title required; description; status restricted to Backlog, In Progress, Completed; estimated_hours numeric, nonnegative; start_date and due_date required; due_date >= start_date.
TASK_ASSIGNMENT	(user_id, task_id) composite PK and FKs. Add/remove operations require an authorized leader for the task's team.
DELIVERABLE / TASK_STEP	Each has an ID PK, task_id FK, nonempty description and is_complete boolean. TASK_STEP also has positive step_order unique within a task.
TASK_LINK	link_id PK; task_id FK; url; optional label. Validate a full http/https URL. No comma-separated link lists.
TIME_ENTRY	entry_id PK; task_id FK; user_id FK; worked_on date; hours numeric; created_at and updated_at. All required. hours > 0; sum of a user's entries on one worked_on date <= 24.
TASK_STATUS_EVENT	event_id PK; task_id FK; actor_user_id FK; old_status; new_status; occurred_at. All required. Old and new status differ and use the TASK status values.

Totals and validation

Actual task hours equal SUM(TIME_ENTRY.hours) for that task. A member's actual hours use their user_id and the inclusive worked_on range. Store decimal values, not floating-point hours. Task variance is actual minus estimate; a negative value means below estimate.

Project estimated hours sum each task once, even when several members are assigned. A member's assigned-task estimate includes each assigned task once and is labeled shared where needed. These member estimates must not be added to produce the project estimate. Date selection filters actual work entries; the task estimate remains the whole-task estimate.

New tasks require an estimate, including a valid zero. A legacy/imported task with NULL estimate displays Not estimated and has no numeric variance. An estimate edit cannot turn a valid value into an invalid or missing one. Missing is never treated as zero. The daily hours limit is checked on create and edit across all tasks for the signed-in user, with a transaction/lock to prevent concurrent entries exceeding 24 hours.

Normalization and verification

Each table stores facts about its key. Memberships and assignments are separate rows, task links and steps are atomic, and actual totals are derived rather than duplicated on TASK. Names and team roles are not copied into time entries. The old TASK.actual_hours and TASK.doc_url fields are replaced by derived totals and TASK_LINK records. Migrate existing values only when their attribution is known; do not invent users or worked dates for a legacy total.

Design checks map to UAT A1 (roles), A2/A4 (tasks and estimates), M2 (entries and totals), M1.7-M1.8 (status records), and A3 (dashboard sums and access). The diagram specifies the design; implementation and execution results are recorded separately.